

Kubernetes Architecture

Table of Contents

1. Purpose of Kubernetes	2
2. Desired state and the object hierarchy	2
3. Lifetime bands	3
4. Kubernetes objects Spark uses	4
4.1. Cluster and nodes	5
4.2. Namespace	5
4.3. Workload controllers: Deployment and StatefulSet	5
4.4. ReplicaSet	6
4.5. Pod	6
4.6. Container and process	6
4.7. Service	6
4.8. ConfigMap and the rest of the declared furniture	7
5. Other Kubernetes objects	7
6. The Spark Worker Deployment template	7
7. From Kubernetes objects to Dynatrace to ServiceNow	9



Draft chapter. Intended to follow **Spark Architecture** in `Log_to_Incident.adoc`. Not merged into that book. To include later, drop this document header and use `include::Kubernetes_Architecture.adoc[leveloffset=+1]`, or paste the body under a new `== Kubernetes Architecture` chapter.

Kubernetes is the runtime that turns the Spark architecture from the previous chapter into something that actually runs. Spark Master, Spark Worker, and Spark History Server are not processes you start by hand on a host and leave there. They are **workloads** that Kubernetes keeps aligned with a declared desired state: a certain number of Spark Worker processes, a single Spark Master with a stable name, a History Server, and a network identity so clients can find the Master. Dynatrace and ServiceNow both invent a model of that same runtime. Those models only make sense if we are clear about which Kubernetes objects are the long-lived **specification** of Spark, and which objects are the short-lived **instances** Kubernetes creates, replaces, and discards in order to honor that specification.

That distinction is the point of this chapter. Later, in Event Management, we will use it to decide which object an alert or incident should bind to. The short version, which the rest of this chapter prepares: a Spark Worker **pod** is a real running thing, but it is a poor configuration item for service management, because Kubernetes can create and destroy pods faster than ServiceNow can materialize their CMDB records.

1. Purpose of Kubernetes

Kubernetes is a cluster operating system for containerized applications. Its job is not to run one container well. Its job is to **continuously reconcile** what operators declared they want with what is actually running on a pool of machines. You do not tell Kubernetes “start these eight Spark Worker processes on lab2.” You tell it “there should be eight Spark Worker replicas, scheduled on lab2, running this image, with these labels and this configuration.” Kubernetes then creates whatever runtime objects it needs to make that statement true, and it keeps creating, replacing, and rescheduling them as nodes fail, images roll, and replica counts change.

That reconciliation loop is why Spark on Kubernetes looks different from Spark on a static set of VMs. The application still has a Master, Workers, and a History Server. What changes is the **unit of replacement**. On a VM, the host and the Spark process tend to live for weeks. On Kubernetes, the **declaration** (the Deployment or StatefulSet) lives for weeks or months; the **process sandbox** that actually executes Spark (the Pod) may live for minutes. Observability and CMDB models that treat every running sandbox as a first-class, durable configuration item inherit that churn.

2. Desired state and the object hierarchy

Kubernetes organizes work as a tree of API objects. At the top of the tree are objects you create once and change infrequently: the cluster itself, the nodes that provide CPU and memory, and the namespace that scopes Spark away from other applications. In the middle are **workload controllers** — Deployment, StatefulSet, DaemonSet, CronJob — which hold the desired state and a **pod template**. At the bottom, Kubernetes instantiates ReplicaSets, Jobs, Pods, and Containers from those templates. Pods are scheduled onto nodes. Services select pods by label so the rest of the cluster can reach them without knowing pod IP addresses, which change every time a pod is replaced.

The important modeling idea is that **labels do not inherit automatically down this tree**. A label on a Namespace is a scope annotation; Kubernetes does not copy it onto Deployments or Pods. What **does** propagate is the pod template inside a controller: `.spec.template.metadata.labels` is copied onto every Pod the controller creates. ReplicaSets created by a Deployment carry the Deployment’s selector labels. If ServiceNow or Dynatrace is to see a Spark identity on both the workload and the running pods, that identity has to be written in two places in the manifest: on the controller’s own `metadata.labels`, and again on the pod template. The Spark manifests in this repository do exactly that with `servicenow.com/service-instance`.

Figure 1 is the same tree with Spark names filled in, and with color showing how long each kind of object typically lives. Green objects are the declared desired state. Amber objects are the controller’s intermediate revisions. Red objects are the ephemeral runtime.

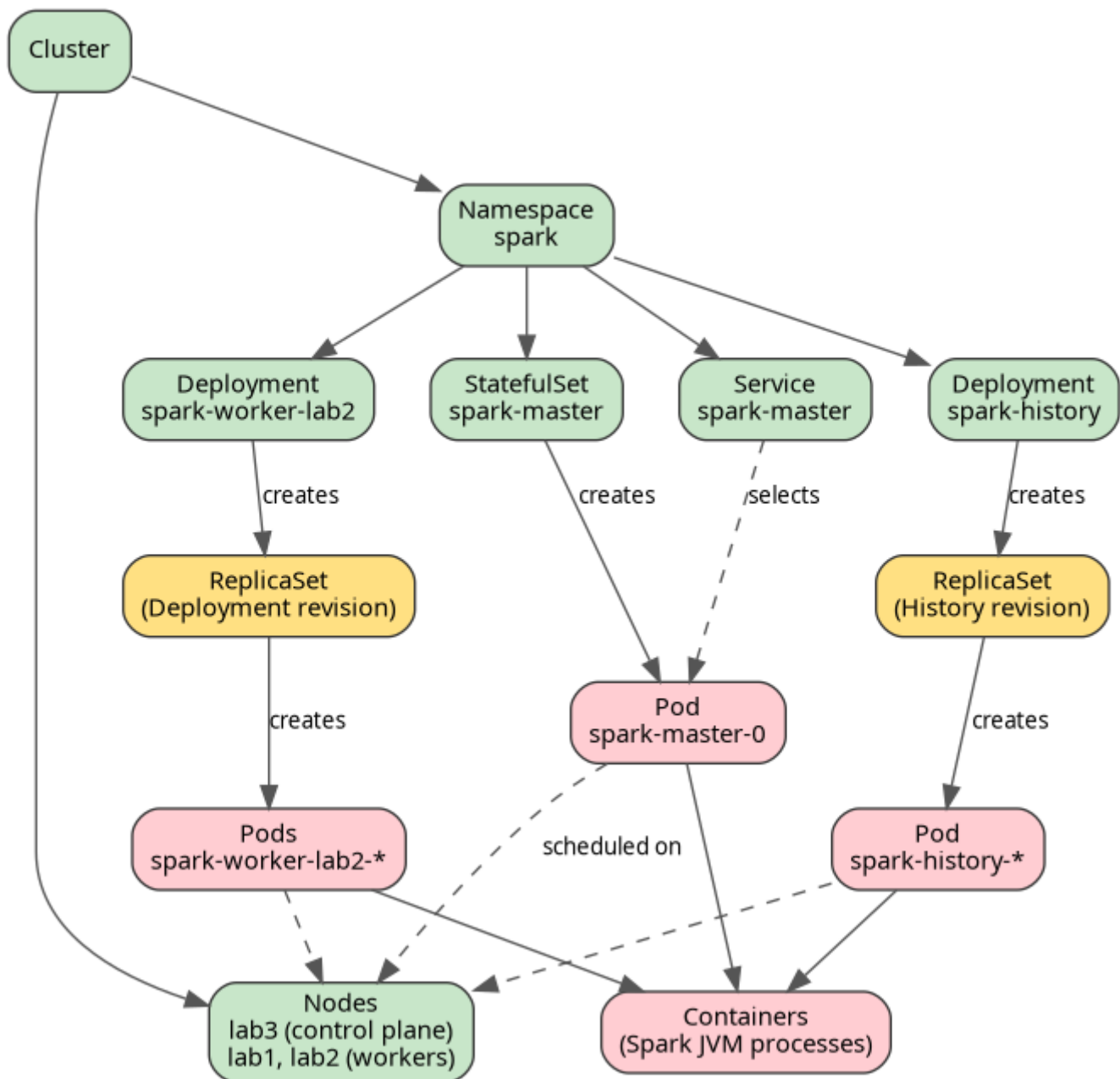


Figure 1. Kubernetes object hierarchy for Spark, colored by lifetime

There **is** an ephemeral level, and it is not a vague property of “the cloud.” It is a specific band of the API tree: **Pod and everything inside it** (containers, and the processes Dynatrace sees inside those containers). Everything above that band is there to **produce** pods. Everything in that band is replaceable. The modeling consequence is that Dynatrace and ServiceNow can usefully represent the green objects as stable entities, must treat the amber objects as implementation detail, and should not depend on the red objects still existing by the time an event has crossed from the cluster into the CMDB.

3. Lifetime bands

Kubernetes objects are not equally durable. For Spark — and for almost any application that scales or rolls — three bands are enough.

The **declared band** is the desired state. The cluster, the nodes, the **spark** namespace, the Spark Master StatefulSet, the Spark Worker Deployments, the History Server Deployment, and the Spark

Master Service are created when Spark is installed and they remain until someone changes the install. Replica counts change; the Deployment object itself does not get a new identity. This is the band that should have a CMDB row for the life of the application.

The **revision band** sits between controller and pod. A Deployment never creates pods directly. It creates a ReplicaSet for the current pod-template hash, and that ReplicaSet creates the pods. A rolling update of the Spark Worker image creates a **new** ReplicaSet, shifts replicas across, and garbage collects the old ReplicaSet. A CronJob (which Spark does not use) similarly creates a Job per schedule tick, and the Job creates one or more pods. ReplicaSets and Jobs are real API objects, and ServiceNow Discovery will create CIs for them, but they are not a stable identity for the application. They exist to implement one generation of the template.

The **ephemeral band** is the pod, its containers, and the OS processes inside those containers. A pod is the smallest deployable unit Kubernetes will schedule: one or more containers that share a network namespace, storage volumes, and a unique name. Kubernetes treats pods as disposable. If a Spark Worker container exits, the ReplicaSet creates a replacement pod with a **different** name. If you scale `spark-worker-lab2` from eight replicas to sixteen, eight new pod names appear. If a node is drained, every pod on it is deleted and recreated elsewhere. Pod IP addresses, UIDs, and often names are not durable identifiers. Containers are even shorter-lived: a container restart inside a still-existing pod is a new process in the same sandbox.

Frequency follows from that design. Declared objects are instantiated when Spark is deployed and then updated on a human timescale — hours to months. ReplicaSets are instantiated on each Deployment rollout, so typically days to weeks in a lab, and as often as every pipeline run in a busy environment. Pods are instantiated whenever desired replicas are not met: at deploy time, on every scale event, on every crash, on every OOM kill, on every node preemption, and on every rolling update. In a Spark Worker Deployment of eight replicas, a single image roll replaces eight pods. A crash-looping worker can create and discard pods on a seconds-to-minutes cadence. That is faster than Kubernetes Visibility Agent (KVA) Informer updates and Service Graph Connector imports can reliably create, relate, and tag a `cmdb_ci_kubernetes_pod` row, and it is certainly faster than tag-based Service Mapping can recompute `svc_ci_assoc` membership for each new name.

StatefulSet pods are a mild exception that is easy to over-read. Spark Master is a StatefulSet with `replicas: 1`, so Kubernetes keeps a pod named `spark-master-0`. The **name** is stable. The **pod object** is not: if the Master container dies, Kubernetes deletes that pod and creates a new one that happens to reuse the same name. Dynatrace still sees a new `CLOUD_APPLICATION_INSTANCE`. ServiceNow may merge on name, or it may briefly have a gap. Stability of the name is not the same as immunity to churn.

4. Kubernetes objects Spark uses

The Spark install occupies one namespace on one cluster. The sections below walk that tree from the top down, naming each object the way Kubernetes, Dynatrace, and ServiceNow will later name it.

4.1. Cluster and nodes

A **cluster** is the control plane plus the member machines. In this lab the control plane runs on lab3 and the Spark executor capacity runs on lab1 and lab2. Dynatrace represents the cluster as `KUBERNETES_CLUSTER` and each member as `KUBERNETES_NODE`. ServiceNow stores those as `cmdb_ci_kubernetes_cluster` and `cmdb_ci_kubernetes_node`. The node is not the Linux host; it is Kubernetes' view of that host as a schedulable worker. The Linux host is a separate object — Dynatrace `HOST`, ServiceNow `cmdb_ci_linux_server` — that OneAgent discovers independently. A pod **runs on** a node; the node **runs on** a host. Both views appear in the Spark service-side CMDB model, and they should not be collapsed: Event Management that binds to the host is binding to the machine that tailed a log file, not to the Spark workload that wrote it.

Nodes are durable relative to pods. They change when you add or remove machines, not when Spark scales. They are the wrong CI for a Spark application incident, because many unrelated pods share a node, but they are a legitimate CI for node pressure, disk, and kubelet problems.

4.2. Namespace

A **namespace** is a named scope inside the cluster. Spark's controllers, pods, and Services live in `spark`. Namespaces are how you keep Spark's `spark-master` Service from colliding with some other application's object of the same name. Dynatrace calls this `CLOUD_APPLICATION_NAMESPACE` (also surfaced as a Kubernetes namespace entity). ServiceNow stores it as `cmdb_ci_kubernetes_namespace`.

A namespace is durable and shared. Tagging the namespace and expecting every child to inherit that tag is a ServiceNow Tag Engine convenience, not a Kubernetes behavior. For Spark we tag the workload controllers and their pod templates, not the namespace, so that Spark Worker, Spark Master, and Spark History Server can be three different service instances inside the same namespace.

4.3. Workload controllers: Deployment and StatefulSet

A **Deployment** is the usual controller for stateless, horizontally scaled software. It stores a replica count and a pod template. Kubernetes makes the number of matching pods equal the replica count, through ReplicaSets, and it does so without caring about pod names or which node a replica landed on. Spark Workers are Deployments: `spark-worker-lab1` and `spark-worker-lab2`, each with eight replicas pinned to that node by affinity. Spark History Server is also a Deployment, with a single replica. Dynatrace folds every workload controller into one entity type, `CLOUD_APPLICATION`. ServiceNow keeps the Kubernetes kind: `cmdb_ci_kubernetes_deployment` for the Workers and History Server.

A **StatefulSet** is the controller you use when identity matters. Each replica gets a stable ordinal name (`spark-master-0`), a stable network identity through a headless Service, and, if you attach them, stable volumes. Spark Master is a StatefulSet with one replica because clients and Workers find it at a known name and port, and because there is only one Master. Dynatrace still calls this `CLOUD_APPLICATION`. ServiceNow stores it as `cmdb_ci_kubernetes_statefulset`. That class difference matters when Event Management looks up a CI by table: a Master problem must not be required to land in the Deployment table.

Both controllers are the **specification** of a Spark component. They change when operators change the install — image, replica count, labels, resource limits — not when a JVM crashes. `CLOUD_APPLICATION / cmdb_ci_kubernetes_deployment` (or StatefulSet) is therefore the natural durable handle for “this is the Spark Worker workload.”

4.4. ReplicaSet

A **ReplicaSet** guarantees that a given pod template is running N times. You almost never create one by hand. The Deployment controller creates a ReplicaSet per template hash and uses it as a generation. ServiceNow Discovery will create `cmdb_ci_kubernetes_replicaset` CIs. Dynatrace does not give ReplicaSets their own first-class entity; they disappear into the parent `CLOUD_APPLICATION`. That is a hint. ReplicaSets are how Deployments **implement** desired state, not how operators **name** Spark. Modeling or binding at the ReplicaSet layer inherits every rollout as a new identity.

4.5. Pod

A **Pod** is a running sandbox: one or more containers, a unique UID, a name, an IP, and a node assignment. It is the object `kubectl get pods` lists, the directory name under `/mnt/spark/logs/`, and the thing that disappears when Spark Worker is scaled down. Dynatrace calls each pod `CLOUD_APPLICATION_INSTANCE`. ServiceNow stores it as `cmdb_ci_kubernetes_pod`.

Pods are the ephemeral level. They are also the most tempting CI, because they are what is actually on fire when a container OOMs or a log line is written. The temptation is what later Event Management has to resist. A problem that names a pod that KVA has not yet inserted, or has already deleted, cannot bind in ServiceNow. A problem that names a Deployment can, because the Deployment CI was created when Spark was installed and is still there after the pod is gone.

Spark Master’s `spark-master-0` name looks like a host name. It is a StatefulSet ordinal. Treat it as a convenient label on an otherwise ordinary pod, not as a VM.

4.6. Container and process

Inside the pod, Kubernetes runs **containers** from the image in the template. The Spark Worker container executes `start-worker.sh`, which is a JVM. Dynatrace OneAgent, looking from inside the node, also sees a `PROCESS_GROUP / PROCESS_GROUP_INSTANCE` for that JVM. ServiceNow may import those as `cmdb_ci_group` and `cmdb_ci_appl`. They are not Kubernetes API objects. They are the process-level view of the same running worker. They churn with the pod: a new pod is a new process instance. They belong in topology. They are not a better bind target than the pod; they are a more granular one.

4.7. Service

A Kubernetes **Service** is a stable network identity in front of a changing set of pods. It selects pods by label and gives them a DNS name and a cluster IP (or a NodePort, or a headless DNS record). Spark Master is reached as `spark-master.spark.svc.cluster.local:7077` because a Service named `spark-master` selects pods labeled `app: spark-master`. There is also a headless Service for the StatefulSet and a NodePort Service for clients outside the cluster. Spark Workers do not need a

Service of their own; they register with the Master. Dynatrace represents Services as `KUBERNETES_SERVICE`; ServiceNow as `cmdb_ci_kubernetes_service`.

Services are durable and belong in the declared band. They are how other workloads find Spark, not how Spark runs. An incident about "Workers cannot reach Master" may involve the Service; an incident about a Worker log line does not.

4.8. ConfigMap and the rest of the declared furniture

The Worker template also mounts ConfigMaps (`spark-configmap`, `spark-defaults-conf`), uses a ServiceAccount, and attaches hostPath volumes for logs, events, data, and jars. Those objects are part of desired state. They rarely appear as Dynatrace entities and they are not used as Event Management bind targets for Spark logs. They matter because they show that a Deployment is a **bundle**: identity, replica count, pod template, configuration, and storage, not just a container image.

5. Other Kubernetes objects

Spark is not the whole cluster. Three other controller kinds show up in Dynatrace and ServiceNow even when Spark never declares them, and they follow the same template-versus-instance pattern.

A **DaemonSet** runs one pod per node (or per matching node). Node exporters, kube-state-metrics, and Dynatrace OneAgent are typical DaemonSets. Dynatrace still types the DaemonSet as `CLOUD_APPLICATION` and each per-node pod as `CLOUD_APPLICATION_INSTANCE`. ServiceNow uses `cmdb_ci_kubernetes_daemonset`. The DaemonSet is durable; the per-node pods churn when nodes join or the DaemonSet rolls. Bind to the DaemonSet, not to `node-exporter-xxxx` on lab2.

A **CronJob** is a schedule plus a job template. Each tick creates a **Job**; the Job creates one or more pods, the pods run to completion, and they go away. This is the most aggressive ephemeral pattern Kubernetes offers. A nightly reconciliation that lasts four minutes will produce a Job CI and a pod CI that are already candidates for deletion by the time ServiceNow's next discovery interval runs. Dynatrace types the CronJob as `CLOUD_APPLICATION`. ServiceNow has `cmdb_ci_kubernetes_cron_job` and `cmdb_ci_kubernetes_job`. Spark does not use CronJobs; the lab still has to model them correctly for anything else on the cluster, and they are the cleanest illustration of why instance-level CIs cannot be the service-management identity. Tag the CronJob controller. Treat Jobs and their pods as runtime exhaust.

Jobs that are not owned by a CronJob — one-shot batch — are the same story with no schedule. They are `CLOUD_APPLICATION` for the Job object if Dynatrace surfaces it, and `cmdb_ci_kubernetes_job` in ServiceNow, with pods underneath that should not be the CMDB identity of the batch **service**.

6. The Spark Worker Deployment template

The Spark Worker is the best teaching example because it is a Deployment, it scales, and it is the component whose pods churn. The live template is `ansible/roles/spark/templates/spark-worker.yaml.j2`. The listing below is the lab2 half, reduced to the fields that correspond to the objects in [Figure 1](#).

Listing 1. Spark Worker Deployment (lab2), annotated against Kubernetes objects

```
apiVersion: apps/v1
kind: Deployment ①
metadata:
  name: spark-worker-lab2 ②
  namespace: spark ③
  labels:
    app: spark-worker
    app.kubernetes.io/name: spark-worker
    app.kubernetes.io/component: worker
    app.kubernetes.io/part-of: apache-spark
    servicenow.com/service-instance: Spark-Worker ④
spec:
  replicas: 8 ⑤
  selector:
    matchLabels:
      app: spark-worker
      node: lab2 ⑥
  template: ⑦
    metadata:
      labels:
        app: spark-worker
        node: lab2
        app.kubernetes.io/name: spark-worker
        servicenow.com/service-instance: Spark-Worker ⑧
    spec:
      affinity:
        nodeAffinity: ⑨
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/hostname
                    operator: In
                    values:
                      - lab2
      containers:
        - name: spark-worker ⑩
          image: "{{ spark_image }}:{{ spark_tag }}"
          command: ["/bin/bash", "-c"]
          args:
            - exec /opt/spark/sbin/start-worker.sh spark://spark-
              master.spark.svc.cluster.local:7077
```

① **Workload controller.** `kind: Deployment` is the declared Spark Worker workload. Dynatrace: `CLOUD_APPLICATION`. ServiceNow: `cmdb_ci_kubernetes_deployment`. This object is created once per worker pool (`spark-worker-lab1`, `spark-worker-lab2`) and kept.

② **Controller identity.** `metadata.name` is the stable workload name Dynatrace reports as `k8s.workload.name` and ServiceNow stores as the Deployment CI `name`.

- ③ **Namespace.** Places the Deployment (and every ReplicaSet and Pod it creates) in `spark`. Dynatrace: `CLOUD_APPLICATION_NAMESPACE`. ServiceNow: `cmdb_ci_kubernetes_namespace`.
- ④ **Controller labels.** These labels are **on the Deployment object itself**. They are what KVA writes to `cmdb_key_value` on `cmdb_ci_kubernetes_deployment`. They do not automatically appear on pods. They are required if tag-based Service Mapping is to anchor on the workload CI rather than only on pods.
- ⑤ **Desired count.** Kubernetes instantiates a ReplicaSet whose job is to keep eight pods running. Changing this number creates or deletes pods; it does not create a new Deployment.
- ⑥ **Selector.** The Deployment owns pods that match these labels. The ReplicaSet uses the same selector. A Service, if we had one, would use a similar selector to find those pods.
- ⑦ **Pod template.** This is not a pod. It is the cookie cutter. Every time the ReplicaSet needs another replica, it copies this template, assigns a generated name (`spark-worker-lab2-<replicaset-hash>-<random>`), and asks the scheduler to place the new pod on a node.
- ⑧ **Pod-template labels.** Kubernetes copies these onto each created pod. KVA then writes them to `cmdb_key_value` on `cmdb_ci_kubernetes_pod`. The `servicenow.com/service-instance: Spark-Worker` value is the CSDM display name in the Kubernetes label character set (spaces become hyphens). Master and History Server use the same key with `Spark-Master` and `Spark-History-Server`. Putting the tag only here tags pods, not the Deployment; putting it only on `metadata.labels` tags the Deployment, not the pods. Spark writes it in both places.
- ⑨ **Scheduling onto a Node.** Affinity is how this Deployment stays on lab2. The resulting relationship is Pod **scheduled on** Node, which Dynatrace and ServiceNow both record. Eight Worker pods on lab2 still share one Node CI and one Host CI.
- ⑩ **Container.** The running Spark Worker JVM. Dynatrace also sees this as a process group instance. It dies with the pod.

What the template does **not** name is as important as what it does. There is no pod name. There is no ReplicaSet name. Those are generated at instantiation time, at the frequency of the ephemeral band. The objects you can point to in the manifest months later are the Deployment, the namespace, the Service the Workers **call** (`spark-master.spark.svc.cluster.local`), and the labels that tie this workload to the Spark Worker service instance.

Spark Master, by contrast, is the same story with `kind: StatefulSet` and `replicas: 1`. The template still has controller labels and pod-template labels. The difference is that Kubernetes names the pod `spark-master-0` instead of generating a random suffix. History Server is another Deployment, `replicas: 1`, tagged `Spark-History-Server`. Three controllers, three CSDM service instances, one namespace.

7. From Kubernetes objects to Dynatrace to ServiceNow

Dynatrace does not copy the Kubernetes API one-for-one. It compresses the declared band into a small set of entity types and it treats the pod as an **instance of** a workload. ServiceNow's Kubernetes CMDB, filled by KVA and by the Service Graph Connector for Dynatrace (SGO-Dynatrace), is closer to the API: Deployment and StatefulSet are different classes, and ReplicaSets

exist. The table below is the contract among the three models for the objects this chapter has named. Spark-used objects are listed first; DaemonSet, CronJob, and Job are included because they appear in the same Dynatrace and CMDB taxonomies the moment anything else runs on the cluster.

Table 1. Kubernetes constructs mapped to Dynatrace entities and ServiceNow CMDB classes

Kubernetes object	Spark uses it	Lifetime band	Dynatrace entity	ServiceNow CMDB class
Cluster	Yes	Declared	KUBERNETES_CLUSTER	cmdb_ci_kubernetes_cluster
Node	Yes (lab1, lab2, lab3)	Declared	KUBERNETES_NODE	cmdb_ci_kubernetes_node
Namespace	Yes (spark)	Declared	CLOUD_APPLICATION_NAMESPACE	cmdb_ci_kubernetes_namespace
Deployment	Yes (Workers, History Server)	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_deployment
StatefulSet	Yes (Master)	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_statefulset
DaemonSet	Not Spark (node agents)	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_daemonset
CronJob	No	Declared	CLOUD_APPLICATION	cmdb_ci_kubernetes_cronjob
Service	Yes (Master)	Declared	KUBERNETES_SERVICE	cmdb_ci_kubernetes_service
ReplicaSet	Yes (implicit under Worker and History Deployments)	Revision	(none; folded into CLOUD_APPLICATION)	cmdb_ci_kubernetes_replicaset
Job	No	Revision	CLOUD_APPLICATION (when surfaced)	cmdb_ci_kubernetes_job
Pod	Yes	Ephemeral	CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod
Container / process	Yes (Spark JVM)	Ephemeral	PROCESS_GROUP / PROCESS_GROUP_INSTANCE (and container entities)	cmdb_ci_group / cmdb_ci_appl (and container CIs)
Linux host (not a K8s object)	Yes (nodes)	Declared	HOST	cmdb_ci_linux_server

Read the Dynatrace column as a **spec versus instance** model. Every Spark workload controller — Worker Deployment, Master StatefulSet, History Deployment, and, elsewhere, a DaemonSet or CronJob — is one **CLOUD_APPLICATION**. Every pod those controllers create is a **CLOUD_APPLICATION_INSTANCE** of that application. Davis problems about a workload therefore carry **dt.entity.cloud_application**. Problems about a specific sandbox carry **dt.entity.cloud_application_instance**. The first identity survives a crash. The second is the crash.

ServiceNow then has two imports of the same world. KVA watches the Kubernetes API and creates the `cmdb_ci_kubernetes_*` rows, including ReplicaSets that Dynatrace never named. SGO-Dynatrace imports Smartscape entities and IRE-merges them onto those rows when names match, storing the Dynatrace entity id in `sys_object_source`. The merge is why a `CLOUD_APPLICATION` for `spark-worker-lab2` and a KVA Deployment named `spark-worker-lab2` become one CI. It is also why a pod that existed only for forty seconds may appear in Dynatrace, never make it through SGC, and already be gone from the API when KVA next syncs.

Above the Kubernetes classes sits the CSDM model from Phase 1. Apache Spark is the business service. Spark Worker, Spark Master, and Spark History Server are service instances. They are not Kubernetes objects. They are joined to Kubernetes CIs by tag-based Service Mapping on `servicenow.com/service-instance`, which reads `cmdb_key_value` and records membership in `svc_ci_assoc`. If that tag is on the Deployment and the StatefulSet, the service instance is tied to the declared band. If it is only on pods, the service map's membership churns with the ephemeral band: rows appear and vanish as fast as pods do, and an incident that has to walk `svc_ci_assoc` from a pod CI will fail whenever the pod CI is missing.

That is the story this chapter is for. Kubernetes runs Spark by holding a durable specification and instantiating disposable pods from it. Dynatrace already distinguishes those two layers as `CLOUD_APPLICATION` and `CLOUD_APPLICATION_INSTANCE`. ServiceNow can represent both, but the CMDB and Event Management cannot keep up with the instance layer. The next time we talk about binding an event to a CI, the candidate is the workload in the declared band — the Spark Worker Deployment, the Spark Master StatefulSet — not the pod that happened to emit the log line.